Sprint 8 — Facade Pattern

Class 2

Learning Objectives

By the end of this class, we should be able to:

- Compare Facade with Adapter, Mediator, Service Layer, and API Gateway.
- Recognize Facade usage in Java and Spring applications.
- Design multiple focused Facades.
- Build synchronous and reactive Facades.
- Handle failures without turning the Facade into a God Class.
- Apply Facade in modular monoliths and microservices.

Lesson 10 — Facade vs Adapter

We often confuse these two patterns because both sit between a client and other components.

However, they solve different problems.

Adapter

Adapter solves interface incompatibility.

Client
  |
Target Interface
  |
Adapter
  |
External SDK

Example:

```
public interface PaymentGateway {

    PaymentResult pay(PaymentRequest request);
}
```

But ABA SDK provides:

```
public class AbaSdk {

    public AbaResponse submit(
            String account,
            double amount,
            String currency
    ) {
        // External API
        return null;
    }
}
```

The Adapter translates between them.

```
public class AbaPaymentAdapter
        implements PaymentGateway {

    private final AbaSdk abaSdk;

    public AbaPaymentAdapter(AbaSdk abaSdk) {
        this.abaSdk = abaSdk;
    }

    @Override
    public PaymentResult pay(
            PaymentRequest request
    ) {
        AbaResponse response = abaSdk.submit(
                request.account(),
                request.amount().doubleValue(),
                request.currency()
        );
```

```
        return new PaymentResult(
                response.transactionId(),
                response.success()
        );
    }
}
```

Adapter changes the interface.

Facade

Facade simplifies a complex subsystem.

```
Client
  |
BookingFacade
  —— GuestService
  —— AvailabilityService
  —— PricingService
  —— PaymentService
  —— BookingService
  —— NotificationService
```

Example:

```
bookingFacade.book(request);
```

Facade does not translate an incompatible interface.

It provides one simple operation over several services.

Easy Comparison

Adapter	Facade
Solves incompatibility	Solves complexity
Usually wraps one external component	Usually coordinates several components
Changes interface	Simplifies interface
Focuses on translation	Focuses on orchestration

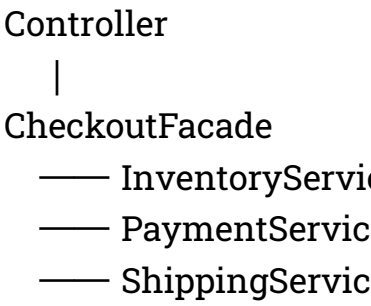
Adapter makes something compatible. Facade makes something simple.

Lesson 11 — Facade vs Mediator

These patterns can look similar because both reduce direct communication between objects.

Facade

Facade provides a simplified entry point for a client.



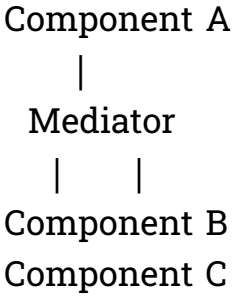
The subsystem classes may still communicate independently.

The main goal is:

Make the subsystem easier for the client to use.

Mediator

Mediator controls communication between collaborating objects.



The components communicate through the Mediator instead of calling each other directly.

The main goal is:

Reduce many-to-many communication between components.

Example

Suppose a booking form contains:

Room Type
Check-in Date
Check-out Date
Guest Count
Price
Booking Button
When one field changes, several others may need updating.

A Mediator can coordinate those UI components.

A Booking Facade is different. It coordinates backend services to complete the booking.

Comparison

Facade	Mediator
Simplifies subsystem usage	Coordinates peer communication
Client calls Facade	Components communicate through Mediator
Usually one-way entry point	Often two-way coordination
Structural Pattern	Behavioral Pattern

Lesson 12 — Facade vs Service Layer

This comparison is important in Spring Boot projects.

Many developers create a class with @Service and automatically call it a Facade.

That is not always correct.

Service Layer

A Service Layer defines the application's available business operations.

Examples:

```
public interface BookingService {

    Booking createBooking(
            CreateBookingCommand command
    );

    Booking cancelBooking(
            String bookingId
    );
}
```

It represents application or business capabilities.

Facade

A Facade simplifies interaction with several services or subsystems.

Example:

```
public class BookingFacade {

    private final GuestService guestService;
    private final AvailabilityService availabilityService;
    private final PricingService pricingService;
    private final PaymentService paymentService;
    private final BookingService bookingService;

    public BookingResult book(
            BookingRequest request
    ) {
        // Coordinate services
        return null;
    }
}
```

Can One Class Be Both?

Yes.

In a small application, one Spring @Service may act as:

- Application Service
- Use Case
- Facade

Example:

```
@Service
public class BookingApplicationService {

    public BookingResult book(
            BookingRequest request
    ) {
        // Coordinate booking workflow
        return null;
    }
}
```

The important question is not the annotation.

The important question is:

What responsibility does the class perform?

Lesson 13 — Facade vs API Gateway

Developers working with microservices often confuse these.

Facade

Facade is an object-oriented design pattern.

It usually exists inside an application.

```
Controller
    |
RoomPublishingFacade
   —— RoomService
   —— MediaService
  —— ModerationService
```

API Gateway

API Gateway is an architectural component.

It sits between external clients and backend services.

Mobile App
Web App
    |
API Gateway
  —— User Service
  —— Room Service
  —— Booking Service
  —— Payment Service

It may handle:

- Routing
- Authentication
- Rate limiting
- Request filtering
- Load balancing
- Logging
- Response aggregation

Relationship

An API Gateway may expose a Facade-like interface to clients.

However, they are not exactly the same concept.

Facade	API Gateway
Design Pattern	Architectural Component
Usually in-process	Usually network boundary
Simplifies objects or services	Routes and protects APIs
Often focused on a use case	Often handles cross-cutting infrastructure

Lesson 14 — Multiple Focused Facades

A large application should not have one Facade for everything.

Bad:

```
public class ApplicationFacade {

    public void registerUser() {
    }

    public void createRoom() {
    }

    public void publishRoom() {
    }

    public void createBooking() {
    }

    public void pay() {
    }

    public void generateReport() {
    }
}
```

This becomes a God Class.

Better Design

RegistrationFacade
AuthenticationFacade
RoomPublishingFacade
BookingFacade
CheckoutFacade
SubscriptionFacade

ModerationFacade
Each Facade represents one focused use case or subsystem.

Example

Booking Facade

```
public class BookingFacade {

    public BookingResult book(
            BookingRequest request
    ) {
        return null;
    }

    public CancellationResult cancel(
            String bookingId
    ) {
        return null;
    }
}
```

Payment Facade

```
public class PaymentFacade {

    public PaymentResult pay(
            PaymentRequest request
    ) {
        return null;
    }

    public RefundResult refund(
            RefundRequest request
    ) {
        return null;
    }
}
```

These Facades are focused and easier to maintain.

Lesson 15 — Facade in Java

Java contains many Facade-like APIs.

Example 1 — Files

Instead of manually working with streams, buffers, and channels:

String content = Files.readString(path);
The Files class hides many lower-level file operations.

It provides a simpler interface.

Example 2 — Collections

Collections.sort(list);
Collections.unmodifiableList(list);
The utility class provides simple operations over collection-related behavior.

Example 3 — Executors

ExecutorService executor =
    Executors.newFixedThreadPool(5);
The client does not need to understand the details of thread pool construction.

Executors provides a simplified creation interface.

This is also factory-like.

A class may demonstrate ideas from more than one pattern.

Lesson 16 — Facade in Spring Boot

Consider a student registration workflow.

Register Student
    ——— Validate Request
    ——— Save Student
    ——— Generate Student ID
    ——— Generate QR Code
    ——— Create LMS Account
    ——— Send Welcome Email
    ——— Notify Teacher

Subsystem Services

```
@Service
public class StudentWriter {

    public Student save(
            RegisterStudentRequest request
    ) {
        System.out.println("Save student");

        return new Student(
                "STUDENT-001",
                request.name(),
                request.email()
        );
    }
}
@Service
public class StudentIdGenerator {

    public String generate() {
        return "PJS-2026-0001";
    }
}
@Service
public class QrCodeService {

    public String generate(
            String studentId
    ) {
```

```
        return "QR-" + studentId;
    }
}
@Service
public class LmsAccountService {

    public String createAccount(
            Student student
    ) {
        return "LMS-" + student.id();
    }
}
@Service
public class EmailService {

    public void sendWelcomeEmail(
            Student student
    ) {
        System.out.println(
                "Send email to " + student.email()
        );
    }
}
```

Facade

```
@Service
public class StudentRegistrationFacade {

    private final StudentWriter studentWriter;
    private final StudentIdGenerator idGenerator;
    private final QrCodeService qrCodeService;
    private final LmsAccountService lmsAccountService;
    private final EmailService emailService;

    public StudentRegistrationFacade(
            StudentWriter studentWriter,
            StudentIdGenerator idGenerator,
```

```
            QrCodeService qrCodeService,
            LmsAccountService lmsAccountService,
            EmailService emailService
    ) {
        this.studentWriter = studentWriter;
        this.idGenerator = idGenerator;
        this.qrCodeService = qrCodeService;
        this.lmsAccountService = lmsAccountService;
        this.emailService = emailService;
    }

    public RegistrationResult register(
            RegisterStudentRequest request
    ) {
        Student student =
                studentWriter.save(request);

        String studentId =
                idGenerator.generate();

        String qrCode =
                qrCodeService.generate(studentId);

        String lmsAccount =
                lmsAccountService.createAccount(student);

        emailService.sendWelcomeEmail(student);

        return new RegistrationResult(
                student.id(),
                studentId,
                qrCode,
                lmsAccount
        );
    }
}
```

Controller

```
@RestController
@RequestMapping("/api/v1/students")
public class StudentController {

    private final StudentRegistrationFacade facade;

    public StudentController(
            StudentRegistrationFacade facade
    ) {
        this.facade = facade;
    }

    @PostMapping
    public RegistrationResult register(
            @RequestBody RegisterStudentRequest request
    ) {
        return facade.register(request);
    }
}
```

The controller knows only one application entry point.

Lesson 17 — Reactive Facade with Spring WebFlux

In Spring WebFlux, the Facade may coordinate multiple reactive operations.

Suppose room publishing requires:

Load Room
Verify Property
Verify Subscription
Verify Media
Submit Room
Create Moderation Case

Reactive Facade

```
@Service
public class ReactiveRoomPublishingFacade {

    private final RoomService roomService;
    private final PropertyClient propertyClient;
    private final SubscriptionClient subscriptionClient;
    private final MediaClient mediaClient;
    private final ModerationClient moderationClient;

    public ReactiveRoomPublishingFacade(
            RoomService roomService,
            PropertyClient propertyClient,
            SubscriptionClient subscriptionClient,
            MediaClient mediaClient,
            ModerationClient moderationClient
    ) {
        this.roomService = roomService;
        this.propertyClient = propertyClient;
        this.subscriptionClient = subscriptionClient;
        this.mediaClient = mediaClient;
        this.moderationClient = moderationClient;
    }

    public Mono<RoomPublishingResult> publish(
            String roomId,
            String ownerId
    ) {
        return roomService.getRequired(roomId)
                .flatMap(room ->
                        verifyRequirements(room, ownerId)
                                .then(
                                        roomService.submit(room.id())
                                )
                )
                .flatMap(submittedRoom ->
                        moderationClient
                                .createCase(
                                        submittedRoom.id()
```

```
                        )
                        .thenReturn(
                                new RoomPublishingResult(
                                        submittedRoom.id(),
                                        submittedRoom.status()
                                )
                        )
                );
    }

    private Mono<Void> verifyRequirements(
            Room room,
            String ownerId
    ) {
        Mono<Void> propertyCheck =
                propertyClient.verifyPublishable(
                        room.propertyId(),
                        ownerId
                );

        Mono<Void> subscriptionCheck =
                subscriptionClient.verifyActive(
                        ownerId
                );

        Mono<Void> mediaCheck =
                mediaClient.verifyFiles(
                        room.mediaIds()
                );

        return Mono.when(
                propertyCheck,
                subscriptionCheck,
                mediaCheck
        );
    }
}
```

Getting Point

The Facade still provides one simple operation:

facade.publish(roomId, ownerId);
The internal workflow is reactive.

The pattern does not depend on whether the implementation is synchronous or reactive.

Lesson 18 — Sequential vs Parallel Calls

In a Facade, some operations must be sequential.

Example:

Reserve Room
    |
Process Payment
    |
Create Booking
Payment should not happen before room reservation succeeds.

Reactive example:

```
return availabilityService.check(request)
        .flatMap(availableRoom ->
                reservationService.reserve(
                        availableRoom,
                        request
                )
        )
        .flatMap(reservation ->
                paymentService.pay(
                        reservation,
                        request.payment()
                )
        )
        .flatMap(payment ->
```

```
            bookingService.create(
                payment,
                request
            )
        );
```

Some calls may be independent and can run together.

Example:

```
Verify Guest
Verify Room
Verify Subscription
return Mono.when(
        guestService.verify(request.guestId()),
        roomService.verify(request.roomId()),
        subscriptionService.verify(request.ownerId())
);
```

The Facade controls the workflow.

Lesson 19 — Error Handling

A Facade should not hide every failure behind a generic error.

Bad:

```
catch (Exception exception) {
    throw new RuntimeException(
            "Something went wrong"
    );
}
```

The client loses useful information.

Better

Use meaningful application exceptions.

```
public class RoomUnavailableException
```

```
        extends RuntimeException {

    public RoomUnavailableException(
            String roomId
    ) {
        super(
                "Room is unavailable: " + roomId
        );
    }
}
public class PaymentFailedException
        extends RuntimeException {

    public PaymentFailedException(
            String message
    ) {
        super(message);
    }
}
```

The Facade may translate low-level exceptions into application-level exceptions.

Example:

```
try {
    return paymentService.pay(request);
} catch (ExternalPaymentException exception) {
    throw new PaymentFailedException(
            exception.getMessage()
    );
}
```

This keeps external details away from clients.

Lesson 20 — Compensation and Partial Failure

This is a senior-level topic.

Suppose booking follows:

```
Reserve Room
    |
Process Payment
    |
Create Booking
```

What happens if:

- Room reservation succeeds
- Payment succeeds
- Booking creation fails

We may need compensation.

```
Booking Creation Failed
    —— Refund Payment
    —— Release Room Reservation
```

Simple Facade Example

```
public BookingResult book(
        BookingRequest request
) {
    RoomReservation reservation = null;
    PaymentResult payment = null;

    try {
        reservation =
                reservationService.reserve(request);

        payment =
                paymentService.pay(
                        request.payment()
                );

        Booking booking =
                bookingService.create(
```

```
                request,
                reservation,
                payment
            );

        return new BookingResult(
            booking.id(),
            "CONFIRMED"
        );

    } catch (RuntimeException exception) {

        if (payment != null
            && payment.successful()) {
            paymentService.refund(
                payment.transactionId()
            );
        }

        if (reservation != null) {
            reservationService.release(
                reservation.id()
            );
        }

        throw exception;
    }
}
```

Important Point

This example is useful for learning, but distributed microservices require more robust patterns, such as:

- Saga Pattern
- Outbox Pattern
- Idempotency

- Retry
- Dead Letter Queue

Facade coordinates the use case.

It does not automatically solve distributed transactions.

Lesson 21 — Microservice API Composition

Imagine a visitor opens a property details page.

The response requires:

Property Service
Room Service
Review Service
Subscription Service
Media Service
A composition Facade may collect the data.

```
@Service
public class PropertyDetailsFacade {

    private final PropertyClient propertyClient;
    private final RoomClient roomClient;
    private final ReviewClient reviewClient;
    private final MediaClient mediaClient;

    public Mono<PropertyDetailsResponse> getDetails(
            String propertyId
    ) {
        Mono<PropertyResponse> property =
                propertyClient.getById(propertyId);

        Mono<List<RoomResponse>> rooms =
                roomClient.getPublishedRooms(
```

```
                propertyId
            );

        Mono<ReviewSummary> reviews =
            reviewClient.getSummary(
                propertyId
            );

        Mono<List<MediaResponse>> media =
            mediaClient.getPropertyMedia(
                propertyId
            );

        return Mono.zip(
            property,
            rooms,
            reviews,
            media
        ).map(tuple ->
            new PropertyDetailsResponse(
                tuple.getT1(),
                tuple.getT2(),
                tuple.getT3(),
                tuple.getT4()
            )
        );
    }
}
```

The client receives one response instead of calling several services.

Facade vs Backend for Frontend

A Backend for Frontend, or BFF, may expose a client-specific Facade.

Example:

Mobile App

```
    |
Mobile BFF
  ——— Property Service
  ——— Room Service
  ——— Booking Service
Admin Portal
    |
Admin BFF
  ——— Moderation Service
  ——— User Service
  ——— Reporting Service
```

A BFF is an architectural pattern, but it often uses Facade-like API composition.

Lesson 22 — Testing a Facade

Because a Facade coordinates several services, unit tests should verify:

- Correct services are called
- Correct order is followed
- Results are passed correctly
- Failures stop the workflow
- Compensation runs when needed

Example Test

```
@ExtendWith(MockitoExtension.class)
class BookingFacadeTest {

    @Mock
    GuestService guestService;

    @Mock
    AvailabilityService availabilityService;
```

```
@Mock
PricingService pricingService;

@Mock
PaymentService paymentService;

@Mock
BookingService bookingService;

@InjectMocks
BookingFacade bookingFacade;

@Test
void shouldCreateBooking() {
    BookingRequest request =
            new BookingRequest(
                    "GUEST-1",
                    "ROOM-1"
            );

    when(
            availabilityService.check(
                    "ROOM-1"
            )
    ).thenReturn(
            new AvailableRoom("ROOM-1")
    );

    when(
            pricingService.calculate(
                    request
            )
    ).thenReturn(
            new BigDecimal("100.00")
    );

    when(
            paymentService.pay(
```

```
                any()
            )
        ).thenReturn(
            new PaymentResult(
                "PAY-1",
                true
            )
        );

        when(
            bookingService.create(
                any()
            )
        ).thenReturn(
            new Booking("BOOKING-1")
        );

        BookingResult result =
            bookingFacade.book(request);

        assertEquals(
            "BOOKING-1",
            result.bookingId()
        );

        verify(guestService)
            .validate("GUEST-1");

        verify(availabilityService)
            .check("ROOM-1");

        verify(paymentService)
            .pay(any());

        verify(bookingService)
            .create(any());
    }
}
```

Lesson 23 — Common Enterprise Mistakes

Mistake 1 — Facade with Too Many Dependencies

A Facade with 20 dependencies may indicate that the use case is too large.

Possible improvements:

- Split the workflow
- Extract policies
- Extract domain services
- Create focused sub-Facades
- Reconsider service boundaries

Mistake 2 — Facade Calling Repositories Directly

Bad:

```
public class BookingFacade {

    private final BookingRepository repository;
    private final PaymentRepository paymentRepository;
    private final RoomRepository roomRepository;
}
```

This may mix coordination and persistence details.

Prefer focused services or use cases:

```
BookingWriter
PaymentProcessor
RoomReservationService
```

Mistake 3 — Reusing One Facade for Unrelated Workflows

Avoid:

```
BookingFacade
    book()
    cancel()
    registerUser()
    uploadImage()
    generateReport()
```

Only related operations belong together.

Mistake 4 — Facade Returning Every Internal Object

The Facade should return a focused result.

Good:

```
BookingResult
```

Avoid exposing:

```
RoomEntity
PaymentSdkResponse
InvoiceDatabaseRow
InternalAuditRecord
```

Mistake 5 — Hiding Important Domain Behavior

Facade should simplify usage, not make the domain invisible.

Do not move all logic into procedural Facade code.

Bad:

```
if (guest.getType().equals("VIP")
        && room.getCategory().equals("SUITE")
        && amount.compareTo(...) > 0) {
    // Complex business rule
```

}
Better:

```
discountPolicy.calculate(
        guest,
        room,
        amount
);
```

The Facade coordinates the policy.

Homework Answer — Hotel Booking Facade

Domain Models

```
public record BookingRequest(
        String guestId,
        String roomId,
        LocalDate checkIn,
        LocalDate checkOut,
        PaymentRequest payment
) {
}
public record BookingResult(
        String bookingId,
        String reservationId,
        String paymentTransactionId,
        BigDecimal totalPrice,
        String status
) {
}
```

Booking Facade

```
public class BookingFacade {
```

```
private final GuestService guestService;
private final RoomAvailabilityService availabilityService;
private final PricingService pricingService;
private final RoomReservationService reservationService;
private final PaymentService paymentService;
private final BookingService bookingService;
private final NotificationService notificationService;
private final AuditService auditService;
private final LoyaltyService loyaltyService;

public BookingFacade(
        GuestService guestService,
        RoomAvailabilityService availabilityService,
        PricingService pricingService,
        RoomReservationService reservationService,
        PaymentService paymentService,
        BookingService bookingService,
        NotificationService notificationService,
        AuditService auditService,
        LoyaltyService loyaltyService
) {
    this.guestService = guestService;
    this.availabilityService = availabilityService;
    this.pricingService = pricingService;
    this.reservationService = reservationService;
    this.paymentService = paymentService;
    this.bookingService = bookingService;
    this.notificationService = notificationService;
    this.auditService = auditService;
    this.loyaltyService = loyaltyService;
}

public BookingResult book(
        BookingRequest request
) {
    Guest guest =
            guestService.getRequired(
                    request.guestId()
```

```
        );

AvailableRoom room =
        availabilityService.check(
                request.roomId(),
                request.checkIn(),
                request.checkOut()
        );

BigDecimal totalPrice =
        pricingService.calculate(
                room,
                request.checkIn(),
                request.checkOut()
        );

RoomReservation reservation =
        reservationService.reserve(
                room,
                request
        );

PaymentResult payment =
        paymentService.pay(
                request.payment(),
                totalPrice
        );

Booking booking =
        bookingService.create(
                guest,
                room,
                reservation,
                payment,
                request
        );

notificationService
```

```
                .sendBookingConfirmation(
                        guest,
                        booking
                );

        auditService.recordBookingCreated(
                booking
        );

        loyaltyService.awardPoints(
                guest,
                totalPrice
        );

        return new BookingResult(
                booking.id(),
                reservation.id(),
                payment.transactionId(),
                totalPrice,
                booking.status()
        );
    }
}
```

Controller

```
@RestController
@RequestMapping("/api/v1/bookings")
public class BookingController {

    private final BookingFacade bookingFacade;

    public BookingController(
            BookingFacade bookingFacade
    ) {
        this.bookingFacade = bookingFacade;
    }
```

```
    @PostMapping
    public BookingResult book(
            @RequestBody BookingRequest request
    ) {
        return bookingFacade.book(request);
    }
}
```

Adding LoyaltyService did not require modifying the controller.

Interview Questions and Answers

1. What problem does Facade Pattern solve?

Facade Pattern provides a simple, unified interface to a complex subsystem. It prevents clients from needing to understand every subsystem class, method, dependency, and calling order.

2. Does Facade replace subsystem classes?

No.

Subsystem classes still perform the real work.

The Facade coordinates and simplifies access to them.

3. What is the difference between Facade and Adapter?

Adapter makes an incompatible interface compatible.

Facade provides a simpler interface to a complex subsystem.

4. What is the difference between Facade and Mediator?

Facade simplifies subsystem usage for a client.

Mediator controls communication between collaborating components.

5. Is every Spring @Service a Facade?

No.

@Service is a Spring stereotype.

A class is acting as a Facade only when it provides a simplified entry point over several underlying components or operations.

6. Can an application have multiple Facades?

Yes.

Large applications should usually have several focused Facades, such as:

RegistrationFacade
BookingFacade
CheckoutFacade
RoomPublishingFacade
ModerationFacade

7. Should a Facade contain business rules?

A Facade may enforce simple workflow decisions, but complex domain rules should normally be delegated to domain objects, policies, validators, or specialized services.

8. Can a Facade call another Facade?

Yes, but it should be done carefully.

Too many Facade-to-Facade calls can create hidden coupling and confusing workflows.

Prefer clear use-case boundaries.

9. Can Facade be used with Adapter?

Yes.

Example:

CheckoutFacade
    |
PaymentGateway
    |
AbaPaymentAdapter
    |
ABA SDK
The Facade coordinates checkout.

The Adapter integrates the payment provider.

10. Can Facade be used with Strategy?

Yes.

The Facade may ask a Strategy or Factory to select behavior.

```
PaymentStrategy strategy =
        strategyFactory.get(
                request.paymentMethod()
        );

strategy.pay(request);
```

11. Can Facade be asynchronous?

Yes.

A Facade can return:

Mono<BookingResult>
CompletableFuture<BookingResult>
or publish events for asynchronous processing.

The pattern is about interface simplification, not execution style.

12. What is the danger of Facade Pattern?

The main danger is creating a God Class that:

- Knows every subsystem
- Contains every business rule
- Handles unrelated workflows
- Has too many dependencies
- Becomes difficult to test and maintain

Facade Pattern Summary

Concept	Meaning
Facade	Simple interface over a complex subsystem
Client	Uses the Facade
Subsystem	Performs specialized operations
Main purpose	Reduce complexity for the client
Facade vs Adapter	Simplicity vs compatibility
Facade vs Mediator	Client access vs peer coordination
Reactive Facade	Same pattern with reactive execution
Microservice Facade	API composition over multiple services
Main danger	God Class

Final Pattern Comparison

Pattern	Main Question
Strategy	Which behavior should execute?
Factory	Which object should be created?
Builder	How should a complex object be constructed?
Template Method	Which workflow steps can subclasses customize?

Observer	How can many components react to an event?
Adapter	How can incompatible interfaces work together?
Facade	How can a complex subsystem be made simple to use?

End of Facade Pattern Sprint